

# C++ Oops interview Questions

1. What are the four main principles of OOP and define each of them ?
2. What is the difference between a class and an object in C++?
3. What is the difference between shallow copy and deep copy in C++?
4. What is constructor overloading in C++?
5. What is a virtual function in C++?
6. Explain the concept of multiple inheritance in C++?
7. What is the difference between public, private, and protected access specifiers?
8. What is friend classes
9. What is a pure virtual function?
10. What is the "diamond problem" in C++ and how is it solved?
11. What are `static` members in C++?
12. What is the difference between a function and an operator overloading?
13. What is the difference between Aggregation, Association and Composition in C++?
14. What is the role of `virtual` keyword in C++?
15. What is the difference between static polymorphism/Compile-time Polymorphism and dynamic/Runtime Polymorphism?
16. What is the difference between a constructor and a copy constructor in C++?
17. What is the use of `friend` keyword in C++?
18. What is an abstract class in C++?
19. What is the role of `this` pointer in C++?
20. What is the difference between function overloading and function overriding in C++?
21. What is a "Deep Copy" constructor, and when would you use it?
22. What is the purpose of `explicit` keyword in C++?
23. Explain RAII (Resource Acquisition Is Initialization) in C++?
24. What is `dynamic_cast` in C++? How does it work?
25. What is an "Interface" ?
26. What are Virtual Destructors and why are they important in C++?
27. What is the difference between an "abstract class" and an "interface"?
28. What is Multiple Inheritance and how does C++ handle ambiguity?
29. What is a "Singleton" class in C++? [Design pattern bs ye wala dekh lena ]
30. What is the difference between Composition and Inheritance in C++?
31. What are the main differences between `struct` and `class` in C++?
32. What is a `constructor` in C++? What are the types of constructors?
33. What is the `const` keyword used for in C++?
34. What are the advantages of using templates in C++?
35. What is the purpose of `namespace` in C++?
36. What are type of inheritance is there ?

## Java

1. What are the differences between JDK, JRE, and JVM?
2. What is the difference between `String`, `StringBuilder`, and `StringBuffer` in Java?
3. What is the difference between `final`, `finally`, and `finalize()` in Java?

## Difference in c++ and java OOPS

### 1. Memory Management

- **Java:**
    - **Automatic Garbage Collection:** Java handles memory management automatically. The **Garbage Collector** takes care of reclaiming memory from unused objects.
    - **No Pointers:** Java does not have explicit pointers (although references are used in a similar way). This eliminates issues like pointer arithmetic and memory leaks.
  - **C++:**
    - **Manual Memory Management:** In C++, you need to explicitly allocate and deallocate memory using `new` and `delete` keywords. Failure to deallocate memory can lead to memory leaks.
    - **Pointers:** C++ allows the use of pointers, which can be manipulated manually, potentially leading to errors like dangling pointers, buffer overflows, etc.
- 

### 2. Multiple Inheritance

- **Java:**
  - **No Multiple Inheritance** (for classes): Java does not support multiple inheritance of classes to avoid ambiguity (the diamond problem). Instead, Java allows **interfaces** to be implemented by multiple classes.
  - **Interfaces:** Java uses interfaces to allow multiple inheritance behavior. A class can implement multiple interfaces.
  - **Example:**

```
java
Copy
interface A { void methodA(); }
interface B { void methodB(); }
class C implements A, B {
    public void methodA() { }
    public void methodB() { }
}
```

- **C++:**

- **Supports Multiple Inheritance:** C++ allows a class to inherit from more than one class. However, it can lead to complications like the diamond problem.
  - **Virtual Inheritance:** C++ provides virtual inheritance to handle issues related to multiple inheritance.
- 

### 3. Constructor Overloading

- **Java:**
    - Java supports **constructor overloading**, allowing multiple constructors with different parameters in the same class.
    - **No Default Constructor in Some Cases:** If no constructor is explicitly defined, the compiler provides a default constructor (without parameters).
  - **C++:**
    - C++ also supports **constructor overloading**. However, C++ **allows default constructors and copy constructors** implicitly if none are provided.
    - **Copy Constructor:** C++ provides a **copy constructor** by default, which is used when a new object is initialized from an existing object.
- 

### 4. Method Overloading and Overriding

- **Java:**
    - **Method Overloading:** Java supports method overloading (methods with the same name but different parameters). It is resolved at compile-time (static polymorphism).
    - **Method Overriding:** Java supports method overriding (redefining methods in subclasses). It is resolved at runtime (dynamic polymorphism).
    - **@Override Annotation:** Java uses the `@Override` annotation to explicitly mark overridden methods.
  - **C++:**
    - **Method Overloading:** C++ also supports method overloading (same name, different parameters). Like Java, this is resolved at compile-time.
    - **Method Overriding:** C++ supports method overriding (same method signature in subclass). However, you need to use the `virtual` keyword to enable runtime polymorphism (dynamic binding).
    - **No @Override Annotation:** C++ does not have an `@Override` annotation.
- 

### 5. Access Modifiers

- **Java:**
  - Java has **four access modifiers**: `private`, `protected`, `default` (no modifier), and `public`.
  - Java also provides the **protected access modifier** to allow access to subclasses and classes in the same package.

- Java does **not** allow access to private members of a superclass directly from a subclass.
  - **C++:**
    - C++ has **three access modifiers**: `private`, `protected`, and `public`.
    - C++ **does not** have a default access modifier (like Java). If no access modifier is specified, members are **private** by default.
    - C++ allows **friend classes** to access private members of a class, which is not possible in Java.
- 

## 6. Abstract Classes and Interfaces

- **Java:**
    - Java has **abstract classes** and **interfaces** to achieve abstraction.
    - A **class** can implement multiple **interfaces** but can inherit from only one **abstract class** (single inheritance).
    - Java **interfaces** can have default methods (with implementation) from Java 8 onwards.
  - **C++:**
    - C++ supports **abstract classes** (classes with at least one pure virtual function) and **pure virtual functions** for abstraction.
    - C++ does not have an equivalent of Java's **interface** but can achieve similar behavior through **abstract base classes** (with pure virtual functions).
    - C++ does not allow **default method implementations** in abstract classes like Java interfaces do.
- 

## 7. Exception Handling

- **Java:**
    - Java has a **checked exception** mechanism where exceptions must either be caught or declared in the method signature using `throws`.
    - Java provides **try-catch-finally** blocks for handling exceptions.
    - **RuntimeExceptions** are unchecked and do not need to be declared or caught.
  - **C++:**
    - C++ uses **try-catch** blocks for exception handling but does not differentiate between checked and unchecked exceptions.
    - C++ does not require methods to declare exceptions they may throw. You can throw any exception without specifying it in the function signature.
- 

## 8. String Handling

- **Java:**
  - Java's **String** class is **immutable**, meaning once a `String` object is created, its value cannot be changed. Any modification creates a new string.

- Java provides a `StringBuilder` and `StringBuffer` for mutable strings.
  - **C++:**
    - In C++, **strings** are mutable by default. `std::string` allows modification of string values.
    - C++ does not have a built-in **immutable string** class.
- 

## 9. Polymorphism

- **Java:**
    - **Runtime Polymorphism** (Dynamic Binding): Achieved using method overriding. The decision of which method to call is made at runtime based on the object's actual type.
    - **Compile-time Polymorphism** (Static Binding): Achieved using method overloading and operator overloading.
  - **C++:**
    - **Runtime Polymorphism:** Achieved using **virtual functions** and function overriding. C++ requires the `virtual` keyword to support dynamic polymorphism.
    - **Compile-time Polymorphism:** Achieved using method overloading and operator overloading.
- 

## 10. Operator Overloading

- **Java:**
  - Java does **not support operator overloading**. It focuses on simplicity and avoids potential ambiguity and complexity.
- **C++:**
  - C++ allows **operator overloading**, enabling developers to define or change the meaning of operators like `+`, `-`, `==`, etc., for user-defined types.